

# MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 19 tháng 6 năm 2026

# Thành Viên Thực Hiện Dự Án

## Hoàng Anh Quân

**MSSV:** 202418969

**Vai trò:** Xây dựng các Pipeline Registers và Xử lý PipeLine Hazard

## Trương Tùng Dương

**MSSV:** 202418885

**Vai trò:** Thiết kế các khối xử lý chức năng trong Datapath

## Nguyễn Minh Hoàng

**MSSV:** 202418904

**Vai trò:** Phát triển Interconnect và Bus Controller, hoàn thiện CPU 8 core

## Bùi Thị Mai Linh

**MSSV:** 20227240

**Vai trò:** Kiểm thử hệ thống & Viết tài liệu

# Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
  - Bối cảnh nghiên cứu
  - Chủ đề của dự án
  - Mục tiêu thiết kế
  - Tại sao chọn Pipeline?
- 2 Kiến Trúc Tập Lệnh (ISA)
  - Tập lệnh cơ bản RV32I
  - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)
  - Cấu trúc tổng thể
  - Các khối xử lý logic chức năng
  - Kiến trúc Pipeline và Logic Điều khiển
- 4 Chiến Lược Kiểm Thử (Testing)
  - Kế hoạch & Trạng thái kiểm thử
  - Kiểm thử Lỗi đơn & Đa nhân
  - Phân tích Waveform

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Bối Cảnh Nghiên Cứu & Lý Do Chọn Đề Tài

## Giới hạn vật lý của CPU lõi đơn (Single-core)

Việc tăng hiệu năng bộ vi xử lý bằng cách tăng tần số xung nhịp đã đạt tới giới hạn vật lý do hiện tượng quá nhiệt và tiêu hao năng lượng cực lớn (**Power Wall, Thermal Wall**).

## Chuyển dịch tất yếu sang Đa lõi (Multi-core)

Tích hợp nhiều lõi xử lý hoạt động độc lập trên cùng một chip để tối ưu hiệu năng tính toán song song.

## Bài toán đặt ra

Giải quyết các vấn đề phức tạp về giao tiếp liên lõi, tranh chấp bus bộ nhớ dùng chung (memory contention) và đồng bộ giữa các luồng xử lý phần cứng.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Chủ Đề Của Dự Án

Dự án tập trung nghiên cứu, thiết kế phần cứng ở mức RTL sử dụng ngôn ngữ mô tả phần cứng **Verilog HDL** để mô phỏng bộ xử lý 8 lõi độc lập:

## Kiến trúc nhân đơn

Mỗi lõi CPU dựa trên kiến trúc tập lệnh mở **RISC-V (RV32I subset)** áp dụng kỹ thuật **pipeline 5 tầng** nhằm tối ưu hóa thông lượng lệnh thực thi.

## Kiến trúc đa nhân

Cả 8 nhân CPU kết nối và sử dụng chung một khối bộ nhớ dữ liệu (**Shared Data Memory**) thông qua một bộ điều khiển Bus và phân xử tập trung.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)



# Mục Tiêu Thiết Kế

Dự án đề ra ba mục tiêu cốt lõi cần đạt được:

- **Thiết kế lõi đơn:** Thiết kế lõi xử lý RISC-V có pipeline 5 tầng hoàn chỉnh, xử lý xung đột lệnh (data/control hazards) một cách tối ưu bằng Forwarding, Stall và Flush.
- **Thiết kế trực bus đa lõi:** Xây dựng cơ chế phân xử công bằng (Round-Robin Arbiter), đảm bảo cả 8 nhân CPU đều có thể ghi/đọc bộ nhớ dùng chung mà không xảy ra xung đột hay deadlock.
- **Mô phỏng chức năng:** Sử dụng bộ công cụ nguồn mở **Icarus Verilog** để chạy mô phỏng hệ thống và **GTKWave** để phân tích giản đồ thời gian (waveform).

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

- Bối cảnh nghiên cứu
- Chủ đề của dự án
- Mục tiêu thiết kế
- Tại sao chọn Pipeline?

## 2 Kiến Trúc Tập Lệnh (ISA)

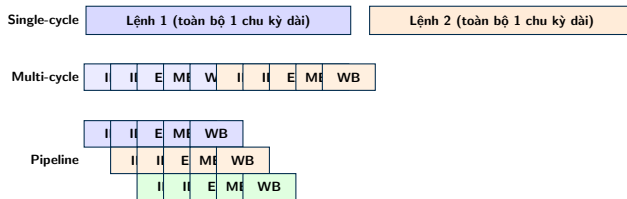
## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Tại Sao Chọn Kiến Trúc Pipeline 5 Tầng?

So sánh 3 kiến trúc thực thi lệnh cơ bản:



## Ưu điểm Pipeline

- Nhiều lệnh **chồng chéo** cùng tồn tại trong CPU ở các tầng khác nhau.
- Thông lượng lý tưởng: hoàn thành **1 lệnh mỗi chu kỳ** (so với 5 chu kỳ ở Multi-cycle).
- Tần số clock cao hơn Single-cycle vì mỗi tầng chỉ chứa một phần logic nhỏ.

## Thách thức cần giải quyết

- **Data Hazard:** Lệnh sau cần kết quả của lệnh trước chưa hoàn tất.
- **Control Hazard:** Branch/Jump làm sai đường thực thi.
- **Structural Hazard:** Nhiều core tranh chấp cùng tài nguyên bộ nhớ.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

- Tập lệnh cơ bản RV32I
- Tập lệnh hiện thực trong dự án

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Tổng quan Tập lệnh RISC-V (RV32I)

## RISC-V là gì?

Là một kiến trúc tập lệnh (ISA) mã nguồn mở dựa trên nguyên lý thiết kế máy tính tập lệnh rút gọn (RISC). Cho phép tự do thiết kế, tùy biến và tối ưu hóa phần cứng.

## Giải nghĩa bộ lệnh cơ bản RV32I:

- **RV (RISC-V):** Tiêu chuẩn kiến trúc thế hệ thứ 5 từ UC Berkeley.
- **32 (32-bit):** Độ rộng thanh ghi và không gian địa chỉ 32-bit.
- **I (Integer):** Tập lệnh số nguyên cơ bản (gồm 40 lệnh). Đây là tập lệnh bắt buộc phải có cho mọi thiết kế chip RISC-V.

## Mục tiêu lựa chọn

Đơn giản hóa thiết kế lõi đơn để tập trung tối ưu bộ điều khiển phân xử (Arbiter) và kết nối đa lõi (8-Core Interconnect).

# Đặc trưng kỹ thuật RV32I

## Các đặc trưng kỹ thuật:

- Không gian địa chỉ và độ dài lệnh cố định **32-bit**.
- Cơ chế **Load-Store**: Chỉ có lệnh Load/Store mới được truy cập Memory; các phép toán toán học chỉ thực thi trên Tập thanh ghi.
- **32 Thanh ghi đa năng** ( $x0 \rightarrow x31$ ), trong đó  $x0$  luôn cố định bằng số 0.
- 6 kiểu định dạng lệnh cơ bản: R, I, S, B, U, J.

## Phân loại Tập thanh ghi (Register File)

- $x0$  (**zero**): Luôn bằng 0 (hằng số phần cứng).
- $x1$  (**ra**),  $x2$  (**sp**): Địa chỉ trả về & Con trỏ Stack.
- $x3-x4$  (**gp-tp**): Con trỏ toàn cục và con trỏ tiểu trình.
- $x10-x17$  (**a0-a7**): Tham số truyền vào & Kết quả hàm.
- $x5-x7$ ,  $x28-x31$  (**t0-t6**): Thanh ghi tạm thời.
- $x8-x9$ ,  $x18-x27$  (**s0-s11**): Thanh ghi lưu giữ.

## 6 Kiểu định dạng lệnh cơ bản

|        |                       |    |    |    |     |     |     |        |        |    |             |        |        |  |
|--------|-----------------------|----|----|----|-----|-----|-----|--------|--------|----|-------------|--------|--------|--|
|        | 31                    | 25 | 24 | 20 | 19  | 15  | 14  | 12     | 11     | 7  | 6           | 0      |        |  |
| R-type | funct7                |    |    |    | rs2 |     | rs1 |        | funct3 |    | rd          |        | opcode |  |
| I-type | imm[11:0]             |    |    |    |     | rs1 |     | funct3 |        | rd |             | opcode |        |  |
| S-type | imm[11:5]             |    |    |    | rs2 |     | rs1 |        | funct3 |    | imm[4:0]    |        | opcode |  |
| B-type | imm[12 10:5]          |    |    |    | rs2 |     | rs1 |        | funct3 |    | imm[4:1 11] |        | opcode |  |
| U-type | imm[31:12]            |    |    |    |     |     |     |        |        | rd |             | opcode |        |  |
| J-type | imm[20 10:1 11 19:12] |    |    |    |     |     |     |        |        | rd |             | opcode |        |  |

### Giải thích các trường

- **opcode**: Mã lệnh gốc (7 bit).
- **rd**: Thanh ghi đích.
- **funct3/7**: Mã mở rộng chức năng lệnh.
- **rs1/rs2**: Các thanh ghi nguồn.
- **imm**: Giá trị hằng số nạp trực tiếp.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

- Tập lệnh cơ bản RV32I
- Tập lệnh hiện thực trong dự án

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)



# Tập lệnh hiện thực trong dự án

## Tập Lệnh Số Nguyên RV32I Subset

40 lệnh cơ bản được hỗ trợ:

- **Tính toán (ALU):** ADD/SUB, SLL/SRL, AND/OR/XOR, SLT (bao gồm cả dạng hằng số Imm như ADDI, ANDI,...).
- **Bộ nhớ:** LW (Load Word), SW (Store Word).
- **Điều khiển luồng:** BEQ, BNE, BLT, BGE, JAL, JALR.
- **Khác:** LUI, AUIPC.

## Đồng Bộ Đa Nhân (A-Extension)

Thiết kế riêng cho hệ thống 8-Core:

- **Độc quyền truy cập:** LR.W (Load-Reserved) và SC.W (Store-Conditional) để xây dựng cấu trúc đồng bộ (Mutex/Semaphore).
- **Phép toán nguyên tử (AMO):** AMOSWAP, AMOADD, AMOAND, AMOOR, AMOXOR, AMOMIN/MAX trực tiếp trên bộ nhớ dùng chung.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

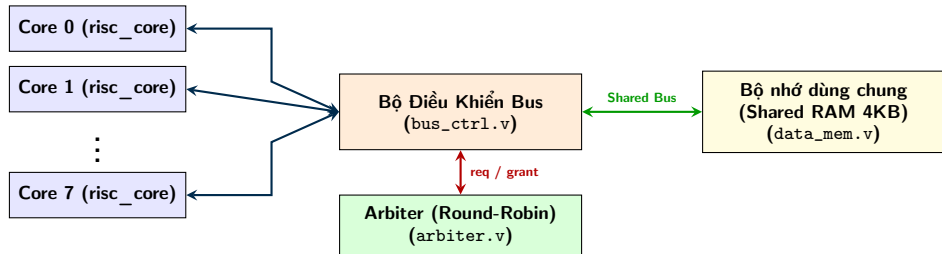
## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Các khối xử lý logic chức năng
  - Bộ nhớ lệnh (Instruction Memory)
  - Khối PC Logic
  - Khối Tập thanh ghi (Register File)
  - Khối Tính toán Số học & Logic (ALU)
  - Bộ nhớ dữ liệu (Data Memory)
- Kiến trúc Pipeline và Logic Điều khiển
  - Thanh ghi đệm Pipeline Registers và Control Unit
  - Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

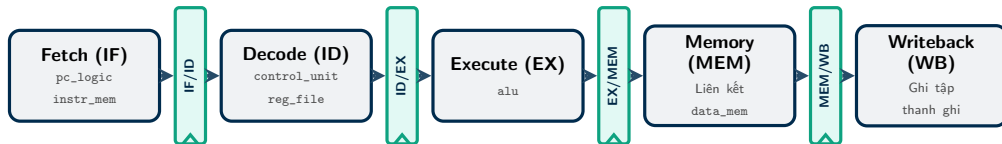
# Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

# Phân bố phần cứng trên 5 tầng Pipeline

Đường truyền dữ liệu (Datapath) ánh xạ trực tiếp các module Verilog vào từng giai đoạn xử lý:



## Nguyên lý hoạt động của Đường dữ liệu

- **Khởi logic chức năng** (pc\_logic, reg\_file, alu,...) thực hiện nhiệm vụ xử lý trong từng tầng riêng biệt.
- **Thanh ghi đệm Pipeline** (IF/ID, ID/EX,...) đóng vai trò chốt dữ liệu giữa các sườn clock, giữ cho lệnh ở các tầng không bị trộn lẫn khi thực thi song song.

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

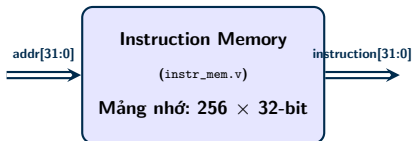
## 3 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Các khối xử lý logic chức năng
  - Bộ nhớ lệnh (Instruction Memory)
  - Khối PC Logic
  - Khối Tập thanh ghi (Register File)
  - Khối Tính toán Số học & Logic (ALU)
  - Bộ nhớ dữ liệu (Data Memory)
- Kiến trúc Pipeline và Logic Điều khiển
  - Thanh ghi đệm Pipeline Registers và Control Unit
  - Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Bộ nhớ lệnh (instr\_mem.v)

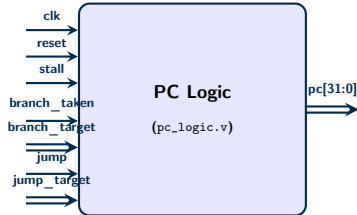
## Sơ đồ khối chức năng



## Mã nguồn Verilog đầy đủ

```
1 module instr_mem #(
2     parameter INIT_FILE = "program.hex",
3     parameter PROGRAM_WORDS = 256
4 ) (
5     input wire [31:0] addr,
6     output wire [31:0] instruction
7 );
8     reg [31:0] mem [0:255];
9     wire [7:0] word_addr = addr[9:2];
10
11     assign instruction = mem[word_addr];
12
13     integer i;
14     initial begin
15         // Khởi tạo NOP trước khi nạp
16         for (i = 0; i < 256; i = i + 1)
17             mem[i] = 32'h0000_0013;
18         $readmemh(INIT_FILE, mem, 0, PROGRAM_WORDS - 1);
19     end
20 endmodule
21
```

## Sơ đồ khối chức năng



## Nguyên lý cập nhật địa chỉ PC

Quy trình cập nhật PC theo mức độ ưu tiên giảm dần:

- 1 **Reset:** Đưa địa chỉ PC về 0 khi bắt đầu.
- 2 **Stall:** Đóng băng PC (giữ nguyên giá trị cũ) khi gặp xung đột dữ liệu hoặc tài nguyên.
- 3 **Jump:** Nhập địa chỉ nhảy không điều kiện ('jump\_target') từ các lệnh 'JAL', 'JALR'.
- 4 **Branch:** Nhập địa chỉ rẽ nhánh ('branch\_target') khi điều kiện rẽ nhánh thỏa mãn (branch\_taken = 1).

# PC Logic: Hiện thực mã nguồn Verilog

## Khai báo Module & Ports

```
1 module pc_logic (  
2     input wire      clk,  
3     input wire      reset,  
4     input wire      stall,  
5  
6     input wire      branch_taken,  
7     input wire [31:0] branch_target,  
8  
9     input wire      jump,  
10    input wire [31:0] jump_target,  
11  
12    output reg [31:0] pc  
13 );  
14
```

## Khởi Logic Cập Nhật PC

```
1     always @(posedge clk or posedge reset) begin  
2         if (reset) begin  
3             pc <= 32'd0;  
4         end  
5         else if (stall) begin  
6             pc <= pc;  
7         end  
8         else if (jump) begin  
9             pc <= jump_target;  
10        end  
11        else if (branch_taken) begin  
12            pc <= branch_target;  
13        end  
14        else begin  
15            pc <= pc + 32'd4;  
16        end  
17    end  
18  
19 endmodule  
20
```



# PC Logic: Thiết lập môi trường Testbench

## Khai báo Tín hiệu & Mạch nạp

```
1 'timescale 1ns/1ps
2
3 module pc_logic_tb;
4
5     reg clk;
6     reg reset;
7     reg stall;
8
9     reg branch_taken;
10    reg [31:0] branch_target;
11
12    reg jump;
13    reg [31:0] jump_target;
14
15    wire [31:0] pc;
16
```

## Khởi tạo UUT & Bộ tạo Xung nhịp

```
1 pc_logic uut (
2     .clk(clk),
3     .reset(reset),
4     .stall(stall),
5     .branch_taken(branch_taken),
6     .branch_target(branch_target),
7     .jump(jump),
8     .jump_target(jump_target),
9     .pc(pc)
10 );
11
12 always #5 clk = ~clk;
13
```

# PC Logic: Kịch bản kiểm thử

## Kịch bản kích thích chính

```
1  initial begin
2      clk = 0; reset = 1; stall = 0;
3      branch_taken = 0; branch_target = 0;
4      jump = 0; jump_target = 0;
5      #10 reset = 0;
6      #20; // Normal PC increment
7      stall = 1; #10; stall = 0; // Stall
8      branch_taken = 1; branch_target = 100;
9      #10; branch_taken = 0; // Branch
10     jump = 1; jump_target = 200;
11     #10; jump = 0; // Jump
12     #20; $finish;
13 end
14
```

## Giám sát tín hiệu & Lưu dạng sóng

```
1  initial begin
2      $monitor(
3          "t=%0t pc=%0d s=%b b=%b j=%b",
4          $time, pc, stall, branch_taken, jump
5      );
6  end
7
8  initial begin
9      $dumpfile("pc_logic_tb.vcd");
10     $dumpvars(0, pc_logic_tb);
11 end
12
13 endmodule
14
```

# PC Logic: Kết Quả Mô Phỏng Console

## Console Output (\$monitor)

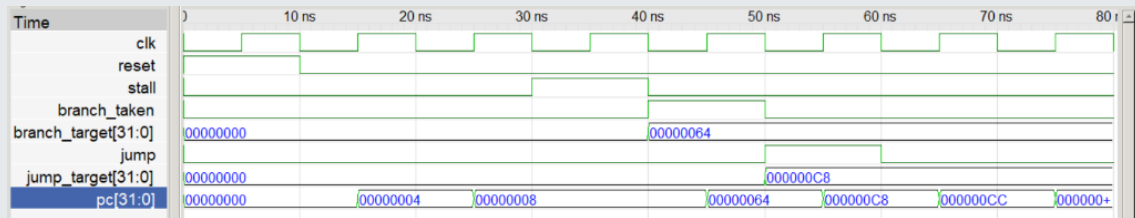
```
Time=0 | PC=      0 | stall=0 | branch=0 | jump=0
Time=15000 | PC=     4 | stall=0 | branch=0 | jump=0
Time=25000 | PC=     8 | stall=0 | branch=0 | jump=0
Time=30000 | PC=     8 | stall=1 | branch=0 | jump=0
Time=40000 | PC=     8 | stall=0 | branch=1 | jump=0
Time=45000 | PC=    100 | stall=0 | branch=1 | jump=0
Time=50000 | PC=    100 | stall=0 | branch=0 | jump=1
Time=55000 | PC=    200 | stall=0 | branch=0 | jump=1
Time=60000 | PC=    200 | stall=0 | branch=0 | jump=0
Time=65000 | PC=    204 | stall=0 | branch=0 | jump=0
Time=75000 | PC=    208 | stall=0 | branch=0 | jump=0
tb/pc_logic_tb.v:63: $finish called at 80000 (1ps)
```

## Phân tích kịch bản kiểm thử

- **0ns – 30ns (Tăng tuần tự):** PC tăng +4 mỗi chu kỳ (0 → 4 → 8).
- **30ns – 40ns (Đóng băng):** stall=1, PC giữ nguyên giá trị bằng 8.
- **40ns – 50ns (Rẽ nhánh):** branch=1, PC nhảy tới 100 (0x64).
- **50ns – 60ns (Lệnh nhảy):** jump=1, PC nhảy tới 200 (0xC8).

# PC Logic: Giải Đồ Sóng Mô Phỏng (GTKWave)

## Giải đồ sóng chi tiết

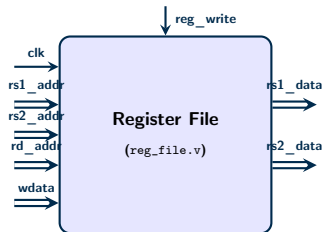


## Nhận xét kết quả trên dạng sóng

- **Thời điểm 30ns (Stall):** Khi stall lên 1, giá trị PC được chốt giữ nguyên ở mức 8 tại sườn lên clock tiếp theo.
- **Thời điểm 40ns (Branch):** Địa chỉ rẽ nhánh branch\_target = 0x00000064 (100) được nạp thành công vào PC.
- **Thời điểm 50ns (Jump):** Địa chỉ nhảy jump\_target = 0x000000C8 (200) được cập nhật thành công vào PC.

# Khối Tập Thanh Ghi (reg\_file.v)

## Sơ đồ khối chức năng



## Nguyên lý hoạt động

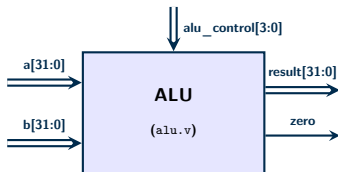
- Quy mô: 32 thanh ghi 32-bit (x0 cứng về 0).
- Đọc bất đồng bộ: rs1\_data, rs2\_data.
- Ghi đồng bộ: rd khi reg\_write=1.
- Cầu bypass: Tránh trễ 1 chu kỳ khi đọc-ghi cùng thanh ghi.

## Mã nguồn Verilog tập thanh ghi

```
1 module reg_file (  
2     input wire      clk, reset,  
3     input wire [4:0] rs1_addr, rs2_addr, rd_addr,  
4     input wire [31:0] write_data,  
5     input wire      reg_write,  
6     output wire [31:0] rs1_data, rs2_data  
7 );  
8     reg [31:0] registers [0:31];  
9  
10    assign rs1_data = (rs1_addr == 5'd0) ? 32'd0 :  
11                      (reg_write && (rd_addr == rs1_addr))  
12                      ? write_data : registers[rs1_addr];  
13    assign rs2_data = (rs2_addr == 5'd0) ? 32'd0 :  
14                      (reg_write && (rd_addr == rs2_addr))  
15                      ? write_data : registers[rs2_addr];  
16  
17    integer i;  
18    always @(posedge clk or posedge reset) begin  
19        if (reset) begin  
20            for (i = 0; i < 32; i = i + 1)  
21                registers[i] <= 32'd0;  
22        end else if (reg_write && (rd_addr != 5'd0)) begin  
23            registers[rd_addr] <= write_data;  
24        end  
25    end  
26 endmodule
```

# Khối Tính Toán Số Học & Logic (alu.v)

## Sơ đồ khối chức năng



## Đặc tả hoạt động

- **Phép toán số học:** ADD (cộng), SUB (trừ), so sánh có dấu SLT.
- **Phép toán logic:** AND, OR, XOR và dịch chuyển bit logic (SLL, SRL).
- **Cờ Zero:** Tín hiệu ra 1-bit, tích cực mức cao khi kết quả toán học bằng 0 (dùng cho lệnh rẽ nhánh).

## Hiện thực khối Case ALU

```
1 always @(*) begin
2     case (alu_control)
3         4'b0010: result = a + b; // ADD
4         4'b0110: result = a - b; // SUB
5         4'b0000: result = a & b; // AND
6         4'b0001: result = a | b; // OR
7         4'b0100: result = a ^ b; // XOR
8         4'b0011: result = a << b[4:0]; // SLL
9         4'b0101: result = a >> b[4:0]; // SRL
10        4'b0111: result = ($signed(a) < $signed(b))
11                    ? 32'd1 : 32'd0; // SLT
12        default: result = 32'b0;
13    endcase
14 end
15
16 // Co bao Zero
17 assign zero = (result == 32'b0);
```

# ALU: Kịch bản kiểm thử (Testbench)

## DUT & Khai báo tín hiệu

```
1 module alu_tb();
2     reg [31:0] a, b;
3     reg [3:0] alu_control;
4     wire [31:0] result;
5     wire zero;
6
7     // Khoi tao Device Under Test (DUT)
8     alu dut (
9         .a(a), .b(b),
10        .alu_control(alu_control),
11        .result(result), .zero(zero)
12    );
13
14    localparam AND = 4'b0000;
15    localparam OR = 4'b0001;
16    localparam ADD = 4'b0010;
17    localparam SLL = 4'b0011;
18    localparam XOR = 4'b0100;
19    localparam SRL = 4'b0101;
20    localparam SUB = 4'b0110;
21    localparam SLT = 4'b0111;
```

## Kịch bản kích thích chính

```
1     initial begin
2         $dumpfile("tb/alu_tb.vcd");
3         $dumpvars(0, alu_tb);
4         // Cases: ADD, SUB, AND, SLT (pos/neg)
5         a = 10; b = 20; alu_control = ADD; #10;
6         a = 50; b = 50; alu_control = SUB; #10;
7         a = 32'hAAAA_AAAA; b = 32'h5555_5555;
8         alu_control = AND; #10;
9         a = 15; b = 25; alu_control = SLT; #10;
10        a = -10; b = 5; alu_control = SLT; #10;
11        // Cases: OR, XOR, SLL, SRL
12        a = 32'hF0F0_FFFF; b = 32'h0F0F_0000;
13        alu_control = OR; #10;
14        a = 32'hF0F0_FFFF; b = 32'hFFFF_0000;
15        alu_control = XOR; #10;
16        a = 32'h0000_000F; b = 4; alu_control = SLL;
17        #10;
18        a = 32'hF000_0000; b = 4; alu_control = SRL;
19        #10;
20        $finish;
21    end
```

# ALU: Kết quả mô phỏng Console

Mô phỏng thực thi chính xác toàn bộ 9 kịch bản kiểm thử trên Console:

## Kết quả xuất ra màn hình (Console Output)

| Time                                | A (Hex) [Dec]       | B (Hex) [Dec]       | Control | Result [Dec]       | Zero |
|-------------------------------------|---------------------|---------------------|---------|--------------------|------|
| 10                                  | 0x0000000a ( 10)    | 0x00000014 ( 20)    | ADD     | 0x0000001e ( 30)   | 0    |
| 20                                  | 0x00000032 ( 50)    | 0x00000032 ( 50)    | SUB     | 0x00000000 ( 0)    | 1    |
| 30                                  | 0xaaaaaaaa (-1.43G) | 0x55555555 ( 1.43G) | AND     | 0x00000000 ( 0)    | 1    |
| 40                                  | 0x0000000f ( 15)    | 0x00000019 ( 25)    | SLT     | 0x00000001 ( 1)    | 0    |
| 50                                  | 0xffffffff6 (-10)   | 0x00000005 ( 5)     | SLT     | 0x00000001 ( 1)    | 0    |
| 60                                  | 0xf0f0ffff (-252M)  | 0x0f0f0000 ( 252M)  | OR      | 0xffffffff (-1)    | 0    |
| 70                                  | 0xf0f0ffff (-252M)  | 0xffff0000 (-65K)   | XOR     | 0xf0f0ffff ( 252M) | 0    |
| 80                                  | 0x0000000f ( 15)    | 0x00000004 ( 4)     | SLL     | 0x000000f0 ( 240)  | 0    |
| 90                                  | 0xf0000000 (-268M)  | 0x00000004 ( 4)     | SRL     | 0x0f000000 ( 251M) | 0    |
| ----- ----- ----- ----- ----- ----- |                     |                     |         |                    |      |
| Kiem tra ket thuc.                  |                     |                     |         |                    |      |

### Nhận xét kết quả:

- **Phép toán số học & logic:** Cộng ('ADD'), trừ ('SUB'), 'AND', 'OR', 'XOR' cho kết quả chính xác 100%.
- **So sánh có dấu SLT:** Kiểm chứng đúng cho cả số dương và số âm ( $-10 < 5$  ra kết quả bằng '1').
- **Dịch chuyển bit SLL/SRL:** Dịch chuyển đúng số bit được yêu cầu (ví dụ: dịch trái '0xF' đi 4 bit ra '0xF0').
- **Cờ Zero:** Tự động phản hồi lên '1' khi kết quả tính toán bằng 0 (như ở phép 'SUB' và 'AND').



# ALU: Giải Đồ Sóng Mô Phỏng (GTKWave)

## Giải đồ sóng chi tiết thu được từ bộ mô phỏng

| Time             | 0        | 10 ns    | 20 ns    | 30 ns    | 40 ns    | 50 ns    | 60 ns    | 70 ns    | 80 ns    | 90 |
|------------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----|
| a[31:0]          | 0000000A | 00000032 | AAAAAAAA | 0000000F | FFFFFFF6 | F0F0FFFF |          | 0000000F | F0000000 |    |
| b[31:0]          | 00000014 | 00000032 | 55555555 | 00000019 | 00000005 | 0F0F0000 | FFFF0000 | 00000004 |          |    |
| alu_control[3:0] | 2        | 6        | 0        | 7        |          | 1        | 4        | 3        | 5        |    |
| result[31:0]     | 0000001E | 00000000 |          | 00000001 |          | FFFFFFFF | 0F0FFFFF | 000000F0 | 0F000000 |    |
| zero             |          |          |          |          |          |          |          |          |          |    |

### • Tín hiệu ngõ vào (Inputs)

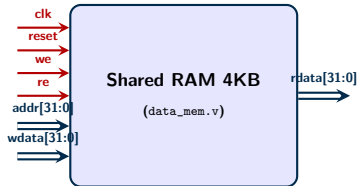
- a[31:0], b[31:0]: Toán hạng ngõ vào (dạng Hex).
- alu\_control[3:0]: Mã lệnh điều khiển (2=ADD, 6=SUB, 0=AND, 7=SLT, 1=OR, 4=XOR, 3=SLL, 5=SRL).

### • Tín hiệu ngõ ra (Outputs)

- result[31:0]: Kết quả tính toán (dạng Hex).
- zero: Cờ báo bằng không, mức cao (1) khi result = 0.

# Bộ Nhớ Dữ Liệu Dùng Chung (data\_mem.v)

## Sơ đồ khối chức năng



## Cơ chế hoạt động bộ nhớ

- **Căn lề địa chỉ (Word-aligned):** Lấy 10 bit địa chỉ thực tế `addr[11:2]` để lập chỉ mục 1024 từ nhớ (mỗi từ 32-bit).
- **Đọc tổ hợp (Bất đồng bộ):** Trả về `rdata` lập tức khi đổi địa chỉ và `re=1` (thuận tiện cho kết nối Bus).
- **Ghi đồng bộ:** Cập nhật mảng nhớ tại cạnh lên `clk` khi `we=1`.

## Mã nguồn RAM 4KB dùng chung

```
1 module data_mem (  
2     input  wire          clk, reset,  
3     input  wire [31:0]  addr, wdata,  
4     input  wire          we, re,  
5     output reg  [31:0]  rdata  
6 );  
7     reg [31:0] mem [0:1023];  
8     wire [9:0] word_addr = addr[11:2];  
9  
10    always @(*) begin  
11        if (re) rdata = mem[word_addr];  
12        else   rdata = 32'd0;  
13    end  
14  
15    integer i;  
16    always @(posedge clk or posedge reset) begin  
17        if (reset) begin  
18            for (i = 0; i < 1024; i = i + 1)  
19                mem[i] <= 32'd0;  
20        end else if (we) begin  
21            mem[word_addr] <= wdata;  
22        end  
23    end  
24 endmodule  
25
```

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

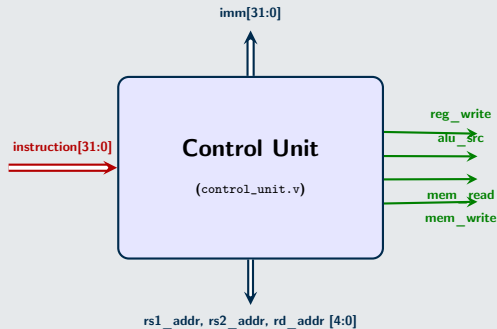
- Cấu trúc tổng thể
- Các khối xử lý logic chức năng
  - Bộ nhớ lệnh (Instruction Memory)
  - Khối PC Logic
  - Khối Tập thanh ghi (Register File)
  - Khối Tính toán Số học & Logic (ALU)
  - Bộ nhớ dữ liệu (Data Memory)
- Kiến trúc Pipeline và Logic Điều khiển
  - Thanh ghi đệm Pipeline Registers và Control Unit
  - Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

# Control Unit

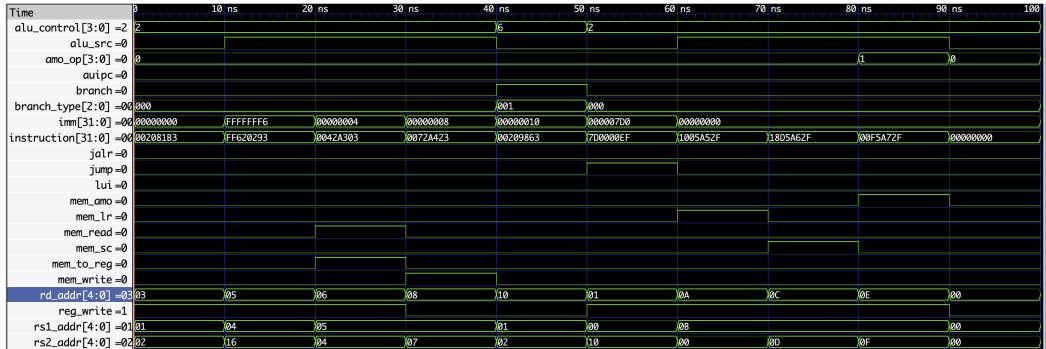
- **Nhiệm vụ:** Sử dụng Combinational Logic để giải mã các trường Opcode, Funct3, Funct7 của Instruction và lập tức sinh các tín hiệu điều khiển cho hệ thống Datapath.

## Sơ đồ Input Output Control Unit



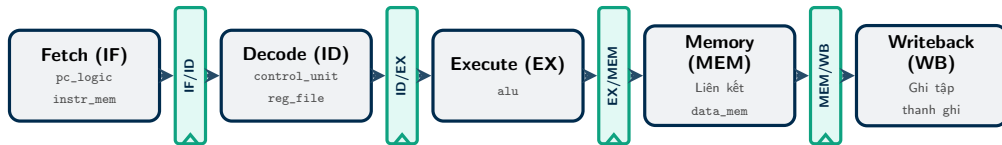
# Unit Test - Control Unit

- Nạp các mã lệnh RISC-V thực tế dưới dạng 32-bit instruction
- Kiểm tra quá trình giải mã opcode, funct3 và funct7
- Quan sát các tín hiệu điều khiển được sinh ra bởi Control Unit
- Xác minh các trường rs1, rs2, rd và immediate được tách chính xác
- Bao phủ nhiều nhóm lệnh: Arithmetic, Load/Store, Branch, Jump và Atomic



# Phân bố phần cứng trên 5 tầng Pipeline

Hệ thống **Pipeline Registers** đóng vai trò là các vách ngăn đồng bộ, cô lập dữ liệu giữa các giai đoạn xử lý:



## Nguyên lý điều khiển luồng trên các Thanh ghi đệm Pipeline

- **Clock Synchronization**: Dữ liệu/lệnh từ tầng trước chỉ được phép dịch chuyển sang tầng sau chính xác tại sườn lên của xung Clock (posedge clk).
- **Pipeline register**: Giữ cố định trạng thái xử lý độc lập của 5 lệnh khác nhau cùng lúc, ngăn chặn hiện tượng lẫn át hay trộn lẫn dữ liệu giữa các tầng Datapath.
- **Pipeline Interlock**: Là nơi trực tiếp tiếp nhận các lệnh (hold để tạo Stall, hoặc clear để tạo Flush/Bubble) từ bộ điều khiển trung tâm.

# PIPELINE REGISTERS: Fetch / Decode (IF/ID Register)

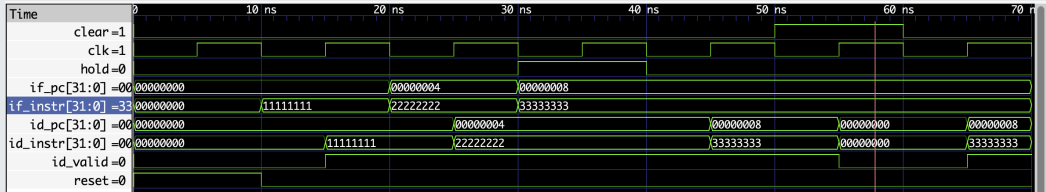
- **Vai trò:** Chốt giữ Instruction 32-bit vừa đọc ra từ Instruction Memory và địa chỉ PC tương ứng để chuyển giao sang giai đoạn Decode.
- **Cơ chế Hazard:** Nhận tín hiệu hold khi kích hoạt Load-Use Stall hoặc Memory Stall để đóng băng trạng thái; nhận tín hiệu clear để thực hiện Flush lệnh rác khi rẽ nhánh Taken sai.

## Sơ đồ Input Output- if\_id.v



# Pipeline Registers Unit Testing - IF/ID

## 1. Kết quả Simulation if\_id



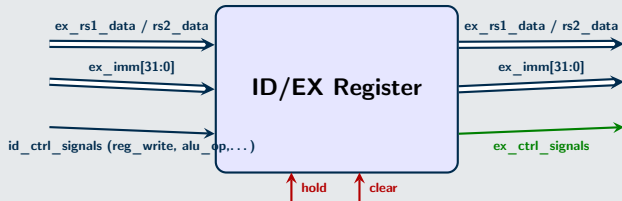
**Phân tích Waveform:** Xác thực phản hồi chính xác khi tín hiệu hold dựng lên, giá trị PC và Instruction được khóa cứng, không bị ảnh hưởng bởi sườn Clock.



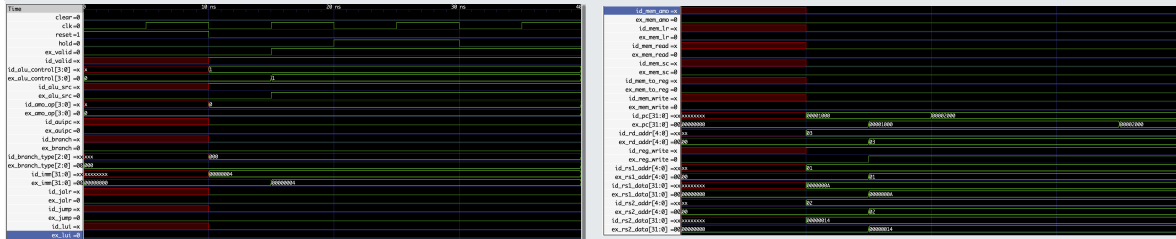
# PIPELINE REGISTERS Decode / Execute (ID/EX Register)

- **Vai trò:** Tiếp nhận và chuyển tiếp toàn bộ các tín hiệu điều khiển từ Control Unit, giá trị các toán hạng đọc từ Register File (`rs1_data`, `rs2_data`) và số Immediate đã mở rộng sang giai đoạn Execute cho ALU.
- **Cơ chế Hazard:** Nhận tín hiệu `clear = 1` để chủ động chèn một Bubble (NOP) vào tầng Execute khi xảy ra xung đột Load-Use.

## Sơ đồ Input Output - id\_ex.v



## 2. Kết quả Simulation id\_ex

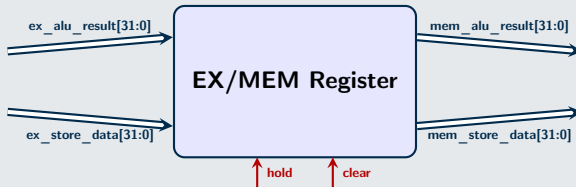


**Phân tích Waveform:** Tín hiệu clear (Flush) kích hoạt thành công, xóa sạch các cờ điều khiển về trạng thái 0 (NOP Bubble) ngay chu kỳ kế tiếp.

# PIPELINE REGISTERS Execute / Memory (EX/MEM Register)

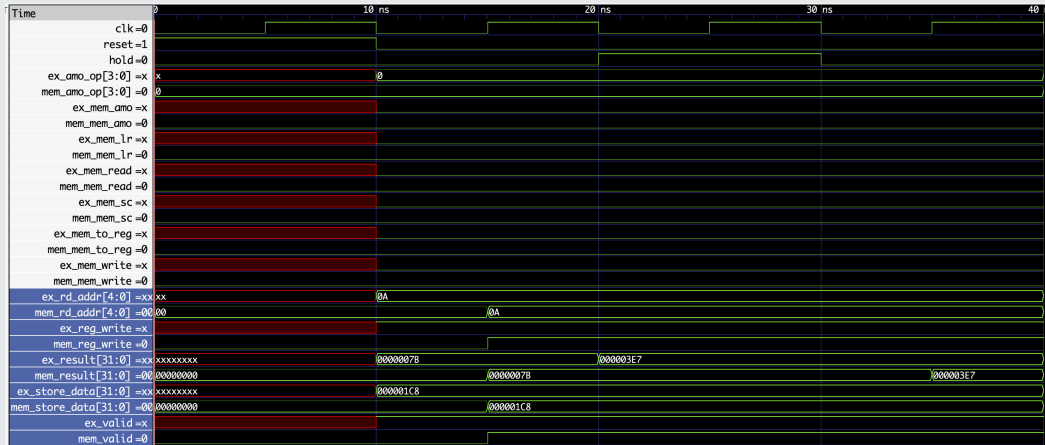
- **Vai trò:** Chốt giữ kết quả tính toán từ đầu ra của ALU, dữ liệu cần ghi vào Data Memory (rs2\_data) của lệnh Store, và địa chỉ thanh ghi đích rd.
- **Cơ chế Hazard:** Nhận tín hiệu hold từ mạch Memory Stall (mem\_wait) nhằm giữ nguyên thông tin khi Shared Bus đang bận phân xử quyền truy cập bộ nhớ đa nhân.

## Sơ đồ Input Output - ex\_mem.v



# Pipeline Registers Unit Testing - EX/MEM

## 3. Kết quả Simulation ex\_mem

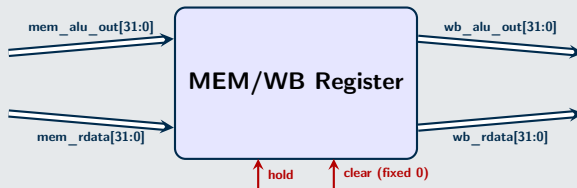


**Phân tích Waveform:** Các toán hạng từ ALU và dữ liệu Store được chốt giữ và chuyển giao đồng bộ

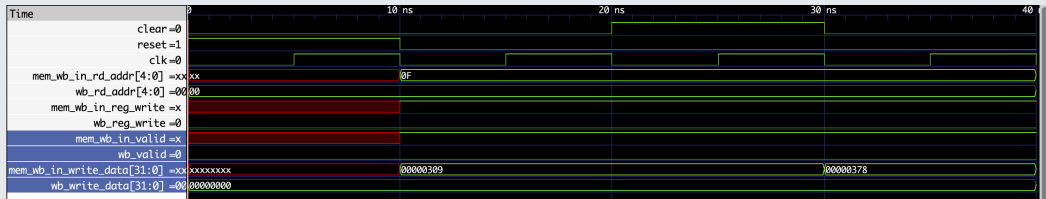
# PIPELINE REGISTERS Memory / Writeback (MEM/WB Register)

- **Vai trò:** Chốt giữ dữ liệu đọc lên từ Data Memory (rdata) hoặc kết quả trực tiếp từ ALU để sẵn sàng đưa về cổng ghi của Register File.
- **Cơ chế Hazard (Đã vá lỗi):** Loại bỏ hoàn toàn chân clear độc hại, chuyển sang sử dụng chân hold đồng bộ. Khi gặp Memory Stall, dữ liệu tầng này đứng im để bảo toàn dữ liệu, tránh lỗi mất thông tin trên Pipeline.

## Sơ đồ Input Output - mem\_wb.v



## 4. Kết quả Simulation mem\_wb

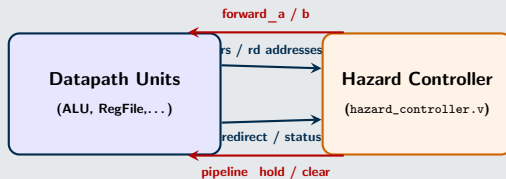


**Phân tích Waveform:** Cơ chế vá lỗi thực tế; sử dụng chân hold thay vì clear. Khi xảy ra Memory Stall (mem\_wait), dữ liệu tầng này đứng im để bảo toàn tính toàn vẹn.

## Nguyên tắc đóng gói:

- Giữ nguyên 100% giao tiếp Ports ra hệ thống Shared Bus ngoại vi và mô hình 8-Core.
- Thực hiện instantiate 6 module con độc lập bên trong ruột và kết nối đồng bộ bằng mạng lưới wire.

## Sơ đồ kết nối khối điều khiển trung tâm trong Core



# Hazard Detection & Forwarding

- **Tư duy tích hợp:** Tập trung hóa (Centralized Control) toàn bộ logic xử lý Load-Use Stall, đi tắt dữ liệu (Forwarding) và quản lý trạng thái bận hệ thống (mem\_wait) vào một bộ điều khiển duy nhất nhằm tối ưu tài nguyên phần cứng.

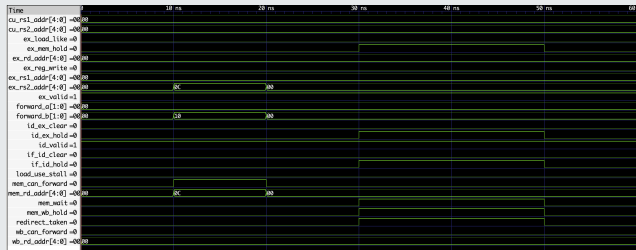
## Sơ đồ khối điều khiển trung tâm hệ thống - Hazard Controller





# Hazard Controller Unit Testing

## Kết quả Simulation trên GTKWave



- Thử nghiệm thành công trên hai kịch bản tối hạn (Critical Scenarios).

## Phân tích định thời Waveform:

### 1. Data Forwarding Interval

Tại khoảng thời gian 10ns - 20ns:

- Cặp cổng giám sát phát hiện ex\_rs2\_addr trùng với mem\_rd\_addr (0x0C).
- forward\_b ngay lập tức phản hồi mã 2'b10, thực hiện đi tắt dữ liệu từ tầng EX/MEM.

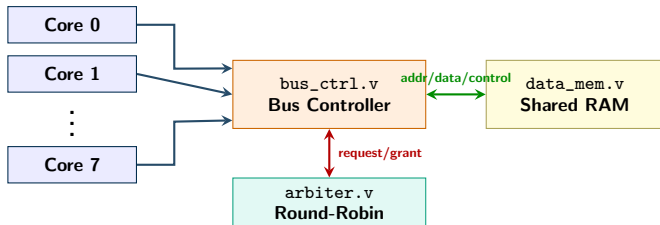
### 2. Memory Stall & Control Flush

Tại khoảng thời gian 30ns - 50ns:

- Khi mem\_wait và redirect\_taken đồng loạt dựng lên mức 1.
- Hệ thống dựng toàn bộ các cờ hold để đóng băng Pipeline.
- Cờ clear bị chặn về 0, bảo vệ an toàn lệnh chờ bộ nhớ khỏi bị xóa nhầm.

# Vai Trò Của Interconnect Trong Hệ Thống 8 Core

Khi 8 core cùng cần đọc/ghi data\_mem, interconnect đóng vai trò cầu nối để chọn đúng core, đúng địa chỉ và trả phản hồi về đúng nơi.



## Nhóm tín hiệu chính

- Core yêu cầu: `core_mem_req`, `core_addr`, `core_wdata`.
- Kiểu giao dịch: `core_we`, `core_re`, `core_lr`, `core_sc`, `core_amo`.
- Phản hồi: `core_ready`, `core_rdata`, `core_sc_result`.

## Ý chính

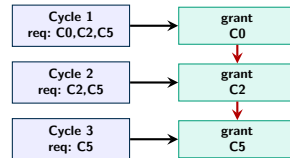
Interconnect không xử lý hazard pipeline; nó cấp quyền truy cập RAM và tạo tín hiệu sẵn sàng để core tiếp tục chạy.

# Arbiter Round-Robin: Chia Quyền Truy Cập Bus

arbiter.v nhận vector request[7:0] và chọn một core được cấp bus trong mỗi lượt cấp quyền.

## Cơ chế cấp quyền

- grant[7:0] là one-hot, chỉ core thắng arbitration được bật bit.
- grant\_valid báo grant hợp lệ trong một chu kỳ.
- grant\_id cho bus\_ctrl biết core nào đang được phục vụ.
- priority\_ptr dịch sang core kế tiếp sau mỗi grant.
- last\_granted tránh cấp lại ngay cho cùng request cũ.

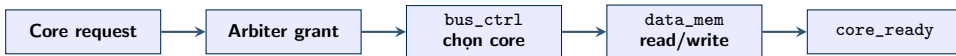


## Mục tiêu

Chia sẻ RAM công bằng hơn giữa các core, giảm nguy cơ một core chiếm bus liên tục.

# Bus Controller: Định Tuyển Giao Dịch Read/Write

Sau khi arbiter cấp quyền, `bus_ctrl.v` chọn tín hiệu của core thắng và điều khiển giao tiếp với `data_mem`.



## Read

- Bật `dm_re` và đưa `dm_addr` sang RAM.
- Nhận `dm_rdata` từ memory interface.
- Ghi dữ liệu vào `core_rdata[grant_id]` và bật `core_ready`.

## Write

- Bật `dm_we`, đưa `dm_addr` và `dm_wdata`.
- `data_mem` ghi dữ liệu theo cạnh clock.
- `bus_ctrl` báo `core_ready` để core kết thúc giao dịch.

Trong thiết kế hiện tại, read ở memory interface là combinational/async; write là synchronous trong `data_mem`.

# Atomic LR/SC Và AMO Trong Interconnect

Interconnect có primitive phần cứng cho đồng bộ đa lõi thông qua reservation và phép toán nguyên tử trên RAM dùng chung.

## LR/SC Reservation

- LR.W: đọc memory và đặt `reservation_valid[core]` tại `reservation_addr[core]`.
- SC.W thành công nếu reservation còn đúng địa chỉ; khi đó RAM được ghi và kết quả trả về là thành công.
- Nếu core khác ghi cùng địa chỉ, reservation bị invalidate; SC.W sau đó thất bại.

## AMO

- AMO đọc giá trị cũ từ RAM.
- `bus_ctrl` tính giá trị mới theo `amo_op`: add, swap, and, or, xor, min/max.
- RAM nhận giá trị mới; core nhận lại old value qua `core_rdata`.

Đây là nền tảng phần cứng để xây dựng cơ chế đồng bộ; slide này không trình bày như một mutex/semaphore phần mềm hoàn chỉnh.

# Testbench Và Kết Quả Kiểm Chứng Interconnect

Kiểm chứng tập trung ở tb/interconnect\_tb.v, tách riêng khỏi pipeline core để kiểm tra trực tiếp arbiter, bus\_ctrl và data\_mem.

| Test          | Mục tiêu kiểm chứng                                               | Kết quả |
|---------------|-------------------------------------------------------------------|---------|
| RR Write/Read | 4 core ghi/đọc các địa chỉ khác nhau qua bus dùng chung.          | PASS    |
| LR/SC success | Core thực hiện LR.W rồi SC.W khi không có core khác ghi chen vào. | PASS    |
| LR/SC fail    | Core khác ghi cùng địa chỉ làm reservation bị xóa trước SC.W.     | PASS    |
| AMOADD        | Hai core cộng nguyên tử, RAM cuối cùng nhận đúng tổng giá trị.    | PASS    |

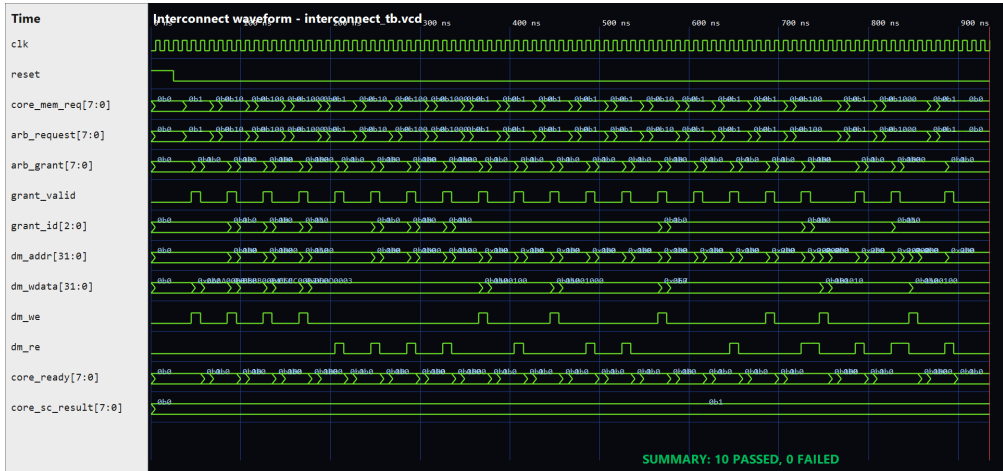
## Kết quả mô phỏng

SUMMARY: 10 PASSED, 0 FAILED

## Tín hiệu kiểm chứng chính

core\_mem\_req, arb\_request, arb\_grant, grant\_id, dm\_addr, dm\_we, dm\_re, core\_ready, core\_sc\_result.

# Waveform Kiểm Chứng Interconnect



Minh chứng luồng core\_mem\_req -> arb\_grant -> dm\_we/dm\_re -> core\_ready và kết quả 10 PASSED, 0 FAILED.

# Phân Tích Kết Quả Kiểm Chứng Interconnect

Dựa trên kết quả mô phỏng từ `interconnect_tb.vcd`, khối interconnect vượt qua 10/10 kiểm tra trong testbench riêng, chứng minh đúng các chức năng chính trong phạm vi đề án:

## 1. Chia sẻ Bus (Round-Robin)

Các core gửi request ghi/đọc qua bus dùng chung. Tín hiệu `arb_grant` cấp quyền theo lượt, cho thấy arbiter chọn đúng core đang yêu cầu và không để một request hợp lệ bị bỏ qua trong test.

## 2. Định tuyến dữ liệu (Bus Control)

Tín hiệu `dm_addr`, `dm_we`, `dm_wdata` được định tuyến từ core được grant tới Shared RAM. Khi giao dịch hoàn tất, `core_ready` bật để báo core nhận được phản hồi.

## 3. Đồng bộ hóa đa lỗi (LR/SC & AMO)

- **Thành công nguyên tử:** Lệnh SC ghi thành công khi không có can thiệp.
- **Phát hiện xung đột:** Khi C0 đang giữ reservation (LR), C1 ghi đè vào địa chỉ đó. Bộ điều khiển lập tức xóa reservation, khiến lệnh SC tiếp theo của C0 **thất bại an toàn**, bảo vệ tính toàn vẹn dữ liệu dùng chung.
- **AMOADD:** Hai phép cộng nguyên tử cập nhật RAM đúng kết quả cuối  $42 + 58 = 100$ .

## Kết Luận

Trong phạm vi testbench, interconnect đã chứng minh được ba nhiệm vụ chính: phân xử bus, định tuyến dữ liệu bộ nhớ và hỗ trợ primitive đồng bộ LR/SC/AMOADD.



# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khôi PC Logic
- Khôi Tập thanh ghi (Register File)
- Khôi Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

- Kế hoạch & Trạng thái kiểm thử
- Kiểm thử Lỗi đơn & Đa nhân
- Phân tích Waveform



# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

- Bộ nhớ lệnh (Instruction Memory)
- Khôi PC Logic
- Khôi Tập thanh ghi (Register File)
- Khôi Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

- Kế hoạch & Trạng thái kiểm thử
- Kiểm thử Lỗi đơn & Đa nhân
- Phân tích Waveform

# Kết Quả Kiểm Thử Lỗi Đơn (Single-Core)

## Kịch bản kiểm thử:

- Nạp file chương trình chứa các lệnh tính toán cơ bản và rẽ nhánh.
- Kiểm tra tính chính xác của Register File và Memory.
- Giám sát các chỉ số tối ưu hóa Pipeline.

## Bảng chứng Pipeline hoạt động

- `load_use_stall = 1`: Có stall khi đọc dữ liệu ngay sau lệnh tải bộ nhớ.
- `flush = 62`: Pipeline giải phóng lệnh thừa khi rẽ nhánh/nhảy.
- `retired = 67`: 67 lệnh hoàn thành chu kỳ ghi kết quả thành công.

## Kết quả chạy Testbench (core\_tb.v)

```
=====
                        CORE TEST RESULTS
=====
x1 (expect 10) = 10 -> PASSED
x2 (expect 20) = 20 -> PASSED
x3 (expect 30) = 30 -> PASSED
x4 (expect 30) = 30 -> PASSED
x5 (expect 1)  = 1  -> PASSED
mem[0] (expect 30) = 30 -> PASSED
```

### PIPELINE EVIDENCE:

`retired=67 mem_stall=4 load_use_stall=1 flush=62`

SUMMARY: 7 PASSED, 0 FAILED

```
=====
```

# Kết Quả Kiểm Thử Hệ Thống 8-Core

## Đánh giá Hiệu năng Hệ thống:

- 8 core chạy song song, ghi/đọc RAM dùng chung không bị deadlock.
- Tín hiệu cấp phát bus của Arbiter công bằng cho tất cả các nhân.

## Chỉ số Giám sát Bus & Pipeline

- Cả 8 core đều có chỉ số grant > 0 (được cấp bus) và retired > 0 (hoàn thành lệnh).
- mem\_stall xuất hiện hợp lý thể hiện độ trễ xếp hàng khi tranh chấp tài nguyên bộ nhớ.

## Kết quả chạy Testbench Hệ thống (system\_tb.v)

```
=====
SYSTEM TEST SUMMARIES
=====
Core 0: retired=16 mem_stall=27 load_use_stall=0 flush=3 grant=4
Core 1: retired=16 mem_stall=29 load_use_stall=0 flush=3 grant=4
Core 2: retired=16 mem_stall=31 load_use_stall=0 flush=3 grant=4
Core 3: retired=16 mem_stall=33 load_use_stall=0 flush=3 grant=4
Core 4: retired=16 mem_stall=35 load_use_stall=0 flush=3 grant=4
Core 5: retired=16 mem_stall=37 load_use_stall=0 flush=3 grant=4
Core 6: retired=16 mem_stall=39 load_use_stall=0 flush=3 grant=4
Core 7: retired=16 mem_stall=41 load_use_stall=0 flush=3 grant=4
```

TỔNG KẾT: 10 PASSED, 0 FAILED

# Nội Dung Báo Cáo

## 1 Giới Thiệu Đề Tài

## 2 Kiến Trúc Tập Lệnh (ISA)

## 3 Thiết Kế Hệ Thống (Design)

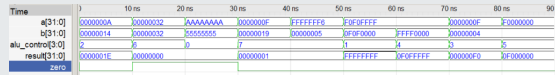
- Bộ nhớ lệnh (Instruction Memory)
- Khối PC Logic
- Khối Tập thanh ghi (Register File)
- Khối Tính toán Số học & Logic (ALU)
- Bộ nhớ dữ liệu (Data Memory)
- Thanh ghi đệm Pipeline Registers và Control Unit
- Bộ phân xử và kết nối đa nhân (Interconnect)

## 4 Chiến Lược Kiểm Thử (Testing)

- Kế hoạch & Trạng thái kiểm thử
- Kiểm thử Lỗi đơn & Đa nhân
- Phân tích Waveform

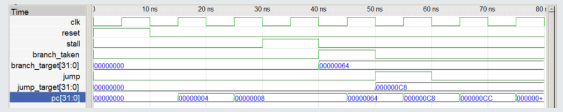
# Minh Chứng Waveform Mô Phỏng

## Kiểm thử khối ALU đơn nhân (alu\_test.png)



- Waveform xác nhận các phép toán cộng, trừ, logic AND/OR hoạt động chính xác tương ứng với các mã opcode từ Control Unit.

## Kiểm thử Logic PC (pc\_test.png)



- Tín hiệu Program Counter tăng tiến tuần tự và rẽ nhánh chính xác khi phát hiện các điều kiện nhảy.